

# Automatización de procesos con n8n para la extracción y gestión de datos públicos: arquitectura C4 del Pipeline RPA Macroentorno Ecuador

Autor: Aaron Robles

Universidad Técnica Particular de Loja - Dirección General de Tecnologías de la Información y Transformación Digital

## I. RESUMEN / ABSTRACT

**Resumen-** La Automatización Robótica de Procesos (RPA) permite convertir tareas digitales repetitivas en flujos controlados, verificables y documentados. Este artículo presenta una revisión aplicada de RPA mediante n8n, tomando como base dos momentos de aprendizaje: un primer reto ya implementado y el diseño arquitectónico del reto final orientado al Tablero Macroentorno Ecuador de la Universidad Técnica Particular de Loja. En el primer reto se construyó un pipeline con tres ramas de consulta: Open-Meteo para información climática, REST Countries para indicadores de país y Open Library para recursos académicos; posteriormente, las ramas se integraron en un reporte final estructurado. En el reto final, aún ubicado en la etapa de arquitectura, se propone automatizar la extracción de datos públicos provenientes del Banco Central del Ecuador, INEC, Superintendencia de Compañías y MINEDUC. El aporte del trabajo consiste en relacionar los fundamentos de RPA con una solución incremental basada en workflows, integraciones HTTP, normalización, validación, manejo de errores, persistencia en Oracle DB y notificación al equipo de Datos.

**Palabras clave-** RPA, n8n, automatización, workflows, APIs, Oracle DB, datos públicos, macroentorno Ecuador.

**Abstract-** Robotic Process Automation (RPA) enables repetitive digital tasks to be transformed into controlled, verifiable, and documented workflows. This paper presents an applied review of RPA using n8n based on two learning stages: a first implemented challenge and the architectural design of the final challenge for the Macroenvironment Dashboard at Universidad Técnica Particular de Loja. The first challenge implemented a three-branch pipeline using Open-Meteo, REST Countries, and Open Library, while the final challenge proposes a scalable architecture for extracting public data from Ecuadorian institutions. The work connects RPA concepts with workflow design, HTTP integrations, data normalization, validation, error handling, Oracle DB persistence, and notifications.

## II. INTRODUCCIÓN

Las organizaciones que trabajan con información pública dependen de fuentes distribuidas en portales, APIs, archivos descargables y servicios web. Aunque esos datos se encuentran disponibles, su actualización manual implica abrir sitios institucionales, revisar si existe una nueva versión, descargar archivos, renombrarlos, validar que no estén vacíos y comunicar el resultado a otros equipos. Este tipo de operación es repetitiva, propensa a errores y difícil de auditar cuando no se registra cada ejecución.

En este contexto, RPA se utiliza como una estrategia para automatizar tareas digitales basadas en reglas. En lugar de que

un usuario repita la misma secuencia de acciones, un workflow puede ejecutar consultas HTTP, guardar archivos, transformar datos, registrar logs y enviar notificaciones. La herramienta n8n resulta adecuada para este escenario porque permite construir flujos visuales con nodos especializados y, cuando es necesario, complementar la lógica mediante código.

El trabajo parte de un primer reto académico ya implementado, en el cual se integraron tres fuentes: Open-Meteo, REST Countries y Open Library. Ese primer pipeline permitió practicar activación manual, consumo de APIs, transformación con nodos Code, consolidación de ramas y generación de un reporte estructurado. Sin embargo, el reto final requiere un alcance mayor: automatizar fuentes públicas reales del macroentorno ecuatoriano y preparar salidas útiles para el equipo de Datos.

El reto final planteado se orienta al Tablero Macroentorno Ecuador. Para ello se consideran fuentes como el Banco Central del Ecuador, INEC, Superintendencia de Compañías y MINEDUC. Cada fuente posee periodicidad y formato distinto; por tanto, el diseño no puede limitarse a un único flujo lineal. Se necesita una arquitectura general que ordene activación, extracción, validación, almacenamiento, manejo de errores, carga posterior y notificación.

La corrección principal del documento consiste en separar adecuadamente el contenido. El estado del arte se resume en una sola sección y se relaciona directamente con el reto final; las herramientas de trabajo, como n8n, Docker, HTTP Request, logs, Oracle Database y C4, se incorporan como parte del marco conceptual; y se agrega una sección propia para describir plenamente el reto, sus fuentes, salidas y criterios de cumplimiento.

El objetivo del artículo es documentar la base conceptual, arquitectónica y metodológica para construir el Pipeline RPA Macroentorno Ecuador. La propuesta no se presenta como una aplicación aislada, sino como un proceso de ingeniería: primero se entiende el problema, luego se diseña la arquitectura mediante C4, después se implementan fuentes de forma incremental y finalmente se valida la trazabilidad mediante archivos, logs, historial de ejecución y notificaciones.

## III. MARCO CONCEPTUAL

### A. Automatización Robótica de Procesos

La Automatización Robótica de Procesos se define como el uso de robots de software para ejecutar actividades digitales repetitivas, estructuradas y basadas en reglas. En ingeniería de procesos, un robot no debe entenderse solo como una simulación de clics, sino como una secuencia controlada que recibe entradas, aplica reglas y produce salidas verificables. Su valor principal está en reducir la variabilidad de tareas manuales y generar evidencia de ejecución.

RPA es adecuada cuando el proceso tiene repetición, reglas relativamente estables y posibilidad de verificación. Estas condiciones se cumplen en el reto final porque las fuentes públicas deben revisarse con periodicidades definidas, los archivos descargados deben guardarse en rutas controladas y cada ejecución debe dejar evidencia de éxito o fallo. La automatización no elimina el análisis posterior del equipo de Datos; se concentra en entregar insumos confiables.

El reto final no busca automatizar una interfaz privada, sino integrar fuentes públicas heterogéneas. Esta característica obliga a tratar la automatización como una capa de integración situada sobre portales institucionales, archivos, respuestas HTTP y base de datos. Por ello, el pipeline debe estar diseñado para soportar cambios moderados en las fuentes, fallos temporales y validaciones mínimas antes de entregar resultados.

La automatización puede ser asistida o desatendida. En el primer reto predominó la ejecución asistida mediante Manual Trigger, porque el objetivo era probar el flujo. En el reto final se requiere avanzar hacia ejecución desatendida con Schedule Trigger, ya que algunas fuentes se actualizan diariamente, otras mensualmente, otras trimestralmente y otras de forma anual o estática.

### B. n8n, workflows, nodos y orquestación

n8n es la plataforma de automatización seleccionada para el reto porque permite construir workflows mediante nodos conectados. Cada nodo cumple una responsabilidad: activar el flujo, configurar una fuente, realizar una solicitud HTTP, validar una condición, transformar datos, guardar archivos o enviar una notificación. Esta separación favorece la modularidad y permite probar cada parte de forma independiente.

Un workflow representa el recorrido lógico de la automatización. En el primer reto se usó un disparador manual, tres ramas de consulta y un nodo de consolidación. En el reto final el patrón se mantiene, pero se amplía: las fuentes se organizan por periodicidad y cada rama debe aplicar reglas comunes de extracción, validación, almacenamiento y registro de errores.

La orquestación consiste en coordinar el orden de ejecución. El orquestador n8n inicia a partir de un evento manual o programado, decide qué fuente ejecutar, envía la solicitud correspondiente y encadena los módulos posteriores. Desde el punto de vista de arquitectura C4, n8n se modela como el contenedor central que coordina el pipeline, no como una lista de nodos aislados.

La modularidad es importante para el mantenimiento. Si una fuente cambia su enlace o formato, se modifica el módulo de extracción o normalización de esa fuente sin rediseñar todo el sistema. Este principio permite que el reto final crezca desde una primera fuente funcional hacia un conjunto de fuentes institucionales.

### C. Triggers, HTTP Request, Code y archivos

Los triggers son eventos de activación. El Manual Trigger se utiliza durante las pruebas, cuando el operador RPA ejecuta el flujo para verificar una fuente. El Schedule Trigger se utiliza

para automatizar ejecuciones según periodicidad: diaria, mensual, trimestral o anual. En el diagrama C4 Nivel 2, ambos triggers aparecen como contenedores de activación conectados al orquestador n8n.

La documentación técnica de n8n respalda esta elección. El Schedule Trigger permite ejecutar workflows en intervalos o fechas específicas, con un comportamiento similar a Cron [19]. El HTTP Request permite realizar solicitudes a servicios o recursos web, por lo que se ajusta a la extracción desde APIs, páginas y enlaces públicos [20]. El Code Node permite ejecutar JavaScript o Python dentro del flujo [21], lo cual es necesario para normalizar campos, construir metadatos, validar respuestas y preparar datos antes de su persistencia.

El nodo HTTP Request es el componente de extracción. Su función es consultar URLs, APIs o recursos publicados por las instituciones. En el reto final debe configurarse para manejar método GET o POST, parámetros, cabeceras, cookies cuando sean necesarias y descarga binaria de archivos. Una respuesta HTTP exitosa no basta; por eso se conecta con un módulo de validación.

El nodo Code permite implementar lógica que no se resuelve de manera directa con nodos visuales. En el reto final puede construir nombres de archivo, agregar fecha de descarga, normalizar campos, verificar existencia de binary.data, generar objetos de log y preparar registros para carga posterior. Esta lógica forma parte de la capa de abstracción del pipeline.

El manejo de archivos es otra herramienta del marco conceptual. Los datos descargados se guardan como archivos crudos en una estructura trazable: institución, categoría y fecha. Esta decisión conserva evidencia de la fuente original y evita depender únicamente de la carga en base de datos. En Docker, la ruta interna usada por n8n se monta hacia una carpeta de Windows para que los archivos sean visibles fuera del contenedor.

### D. Docker, Oracle Database, logs y modelo C4

Docker permite ejecutar n8n en un entorno controlado. Cuando n8n corre dentro de un contenedor, las rutas de Windows no se interpretan directamente; por eso se utiliza un volumen que mapea una carpeta local hacia una ruta interna como /home/node/datos\_macroentorno. Este criterio es fundamental para que el nodo Write File to Disk pueda guardar archivos correctamente.

El modelo C4 se utiliza para documentar la arquitectura de manera gradual. En particular, el diagrama de contenedores muestra la forma general del sistema, las responsabilidades principales y la comunicación entre componentes [22]. Por esta razón, para el reto final se emplea C4 Nivel 2: permite representar triggers, orquestador n8n, extracción, validación, repositorio, normalización, logs, Oracle Database y notificación, sin caer en el detalle de cada nodo interno del workflow.

Oracle Database se considera como destino estructurado posterior. La base de datos no reemplaza al repositorio crudo; lo complementa. Los archivos descargados conservan trazabilidad y la base de datos organiza registros normalizados en tablas staging por fuente. Los logs registran fecha, fuente, estado, intentos, ruta del archivo y mensaje de error.

Finalmente, el modelo C4 se usa para representar la arquitectura de forma general, separando contexto, contenedores y componentes.

Tabla I. Componentes del primer pipeline y conceptos que se reutilizan en el reto final.

| Componente            | Función dentro del flujo                                     |
|-----------------------|--------------------------------------------------------------|
| Inicio de Pipeline    | Activa manualmente la ejecución del workflow.                |
| Open-Meteo            | Consulta clima por ciudad y calcula estadísticas regionales. |
| REST Countries        | Obtiene indicadores básicos de Ecuador, Colombia y Perú.     |
| Open Library          | Filtra recursos académicos sobre automatización.             |
| Merge + Reporte Final | Integra las tres ramas y genera una salida estructurada.     |

#### IV. ESTADO DEL ARTE APLICADO AL RETO FINAL

El estado del arte sobre RPA muestra que la automatización genera valor cuando se aplica a procesos repetitivos, de alto volumen y con reglas claras. En el reto final, la necesidad no consiste en reemplazar un sistema institucional existente, sino en automatizar la obtención de datos desde fuentes externas que publican información con distintos formatos y frecuencias. Por ello, la automatización se entiende como una capa de integración y control.

La literatura reciente permite reforzar esta decisión de diseño. Syed et al. [13] describen que RPA contribuye a mejorar la eficiencia operativa, pero también advierten que su aplicación requiere bases técnicas y organizativas para evitar automatizaciones frágiles. En la misma línea, Wewerka y Reichert [14] presentan una revisión sistemática donde RPA se entiende como automatización de procesos rutinarios basados en reglas. Esta definición se acopla directamente al reto final, porque el pipeline propuesto ejecuta reglas repetibles: consultar fuentes públicas, descargar archivos, validar resultados, registrar evidencias y entregar salidas al equipo de Datos.

Los estudios sobre adopción de RPA también resaltan la necesidad de seleccionar procesos adecuados, documentar criterios de ejecución y controlar excepciones [15]. Por ello, el reto final no se plantea como una automatización improvisada, sino como una arquitectura por contenedores donde existen disparadores, extracción HTTP, validación, logs, repositorio crudo, normalización y notificación. Además, la relación entre RPA y minería de procesos evidencia la importancia de registrar eventos para analizar ejecuciones y fallos [16], aspecto que se refleja en el uso de historial de ejecución, registro de errores y continuidad ante fallos parciales.

Los enfoques actuales de automatización empresarial recomiendan analizar el proceso antes de automatizarlo. Este principio se aplica directamente al reto, porque primero se elaboró una arquitectura C4 Nivel 2 para identificar actores técnicos, triggers, contenedores internos, fuentes públicas, repositorios, logs, Oracle Database y notificación. La arquitectura reduce improvisación y permite justificar cada módulo del workflow.

Las plataformas low-code y no-code, como n8n, aceleran la construcción de integraciones al ofrecer nodos configurables.

Sin embargo, el uso de una herramienta visual no elimina la necesidad de diseño. El flujo debe tener nombres comprensibles, separación por responsabilidades, manejo de errores y documentación. En el reto final, esta disciplina evita que el workflow se convierta en una secuencia difícil de mantener.

La literatura sobre gestión de datos resalta la importancia de trazabilidad y control de calidad. En un pipeline que alimentará un tablero macroentorno, descargar archivos no es suficiente; también se debe registrar origen, fecha, estado, intentos, ruta y errores. Estos metadatos permiten auditar el proceso y diferenciar una actualización exitosa de una ejecución incompleta.

Desde la perspectiva de pipelines de datos, la calidad no depende únicamente de obtener un archivo, sino de controlar integridad, estructura, tipos de datos y comportamiento del flujo. Foidl et al. [17] señalan que los problemas de calidad en pipelines pueden originarse en datos incompletos, estructuras inesperadas o fallos de procesamiento. Esta idea justifica que la arquitectura del reto final incorpore validación antes de guardar, cargar o notificar resultados. A su vez, los enfoques modernos de ETL recomiendan estructuras de transformación reutilizables para integrar fuentes heterogéneas [18], lo que respalda la capa de abstracción propuesta para normalizar datos provenientes de BCE, INEC, Supercias y MINEDUC.

Los trabajos relacionados con pipelines de datos plantean una separación entre extracción, transformación y carga. Aunque el reto se desarrolla con RPA, comparte principios de ETL: extraer desde fuentes públicas, validar la integridad del recurso, normalizar información y preparar una carga posterior. La diferencia es que n8n permite coordinar esas tareas con triggers, nodos HTTP, archivos, código y correo sin desarrollar una aplicación completa desde cero.

El manejo de errores es otro punto clave del estado del arte. Las fuentes públicas pueden fallar por cambios de URL, mantenimiento del servidor, enlaces que no descargan directamente o formatos modificados. Por ello, el diseño del reto final incorpora reintentos, uso de última versión válida si existe, registro de incidente y notificación de fallo parcial sin detener todo el pipeline.

En síntesis, el estado del arte respalda una solución incremental: comenzar con fuentes simples, validar el patrón común y ampliar progresivamente. El valor del reto final no está únicamente en automatizar descargas, sino en establecer una arquitectura trazable, mantenible y alineada con necesidades reales del equipo de Datos.

#### V. DESCRIPCIÓN DEL RETO FINAL

##### A. Alcance institucional y problema a resolver

El reto final consiste en diseñar e implementar progresivamente un pipeline RPA para el Tablero Macroentorno Ecuador. El proceso debe consultar fuentes públicas, descargar archivos o respuestas, validar que los recursos sean útiles, guardarlos en una estructura de carpetas trazable, registrar resultados de ejecución y comunicar el estado al equipo de Datos.

El problema operativo aparece porque la información se encuentra distribuida en distintas instituciones. Cada portal publica sus archivos con reglas propias, nombres diferentes y frecuencias de actualización particulares. Si el proceso se realiza manualmente, aumenta el tiempo invertido, se dificulta la trazabilidad y existe riesgo de trabajar con versiones antiguas o incompletas.

El pipeline no pretende reemplazar el trabajo analítico del equipo de Datos. Su función es preparar insumos confiables: archivos crudos organizados, metadatos de ejecución, logs de error y, en una etapa posterior, datos cargados en Oracle Database. Esta separación permite que el equipo de Datos se concentre en limpieza, transformación y visualización.

El alcance inicial debe ser incremental. Primero se valida una fuente de menor complejidad, como una página o archivo descargable; luego se reutiliza el patrón para las demás. Esta decisión reduce riesgo técnico y facilita demostrar avances semanales durante el reto.

## B. Fuentes públicas y periodicidad

Las fuentes consideradas pertenecen al Banco Central del Ecuador, INEC, Superintendencia de Compañías y MINEDUC. El Banco Central aporta indicadores económicos como cuentas nacionales, sector real, precio de petróleo WTI y riesgo país. INEC aporta datos laborales y censales. Supercias aporta ranking y directorio de compañías. MINEDUC aporta registros administrativos educativos.

La periodicidad es una variable de diseño. Algunas fuentes se actualizan diariamente, como petróleo WTI y riesgo país; otras tienen frecuencia mensual, como ciertos indicadores del sector real o directorios; otras son trimestrales, como ENEMDU; y otras son anuales o estáticas, como cuentas nacionales, censo, ranking empresarial y AMIE.

Organizar el pipeline por periodicidad evita ejecuciones innecesarias. No tiene sentido consultar diariamente un recurso anual, ni esperar un mes para actualizar una fuente diaria. Por ello, el diseño C4 incluye una capa de activación con triggers manuales para pruebas y triggers programados para operación automática.

Cada fuente se trata como una unidad de procesamiento. Para cada una se debe definir nombre lógico, institución, categoría, URL, formato esperado, carpeta destino, periodicidad, reglas de validación y comportamiento en caso de error. Esta configuración permite construir un flujo más uniforme aunque las fuentes sean heterogéneas.

## C. Eventos de activación y funcionamiento esperado

El pipeline puede iniciarse mediante dos tipos de evento. El primero es manual: el operador RPA presiona Execute Workflow en n8n para ejecutar pruebas, validar una fuente o demostrar el funcionamiento al evaluador. El segundo es programado: el planificador n8n o Cron dispara el Schedule Trigger en una fecha y hora definidas.

Cuando el evento se dispara, el orquestador n8n inicia la secuencia de módulos. Primero identifica la fuente o grupo de fuentes que corresponde a la periodicidad. Luego ejecuta el módulo de extracción, que realiza la consulta HTTP. Si la respuesta contiene archivo binario, se prepara el nombre del

archivo con la fecha de descarga y se define la ruta dentro del repositorio trazable.

Después de la extracción se ejecuta la validación. Esta etapa revisa que exista archivo, que no esté vacío, que el formato sea el esperado y que la respuesta no corresponda a una página de error. Si la validación es positiva, el archivo se guarda en el repositorio crudo. Si la validación falla, se activa el manejo de errores.

La etapa de parsing y normalización se encarga de leer CSV, XLSX, ZIP, HTML o JSON, según corresponda. No todas las fuentes tienen el mismo formato, por lo que esta etapa transforma cada respuesta en una estructura común. La normalización puede incluir limpieza de nombres de columnas, conversión de fechas, unidades, códigos de fuente y metadatos de ejecución.

El módulo de consolidación agrupa resultados válidos, metadatos y estados de ejecución. Esta consolidación no significa mezclar todos los datos en una sola tabla; significa producir un resumen operativo que indique qué fuentes fueron procesadas, cuáles fallaron, qué rutas se generaron y qué datos quedan listos para carga posterior.

El resultado esperado se entrega por tres vías. La primera es el repositorio de archivos crudos, que conserva evidencia. La segunda es Oracle Database, donde se proyecta cargar datos normalizados en tablas staging. La tercera es la notificación por correo, que resume la ejecución para el equipo de Datos.

El flujo debe tolerar fallos parciales. Si una fuente no responde, el resto del pipeline puede continuar. Esto es importante porque las instituciones públicas no siempre publican sus recursos con disponibilidad permanente. Un fallo en una fuente no debería invalidar la actualización de las demás.

En la práctica actual, el flujo se está construyendo desde una fuente inicial de Supercias para validar descarga, nombre de archivo, ruta Docker y escritura en disco. Esta implementación parcial permite comprobar el patrón antes de incorporar el resto de las instituciones.

El evento de activación, por tanto, no descarga datos por sí mismo. El trigger solo inicia el flujo. La descarga la realiza el módulo HTTP, la validación la realiza el módulo de control, el guardado lo realiza el sistema de archivos y la notificación la realiza el servicio de correo.

Esta separación facilita explicar el proyecto en términos de arquitectura: un evento inicia un sistema de procesamiento, y cada contenedor cumple una responsabilidad definida.

Tabla II. Patrón técnico heredado del primer reto y reutilizado para construir el reto final.



| Etap              | Nodo principal  | Resultado esperado                                      |
|-------------------|-----------------|---------------------------------------------------------|
| Activación        | Manual Trigger  | Inicio controlado del pipeline.                         |
| Entrada climática | Set + Code      | Lista de ciudades convertida en elementos.              |
| Iteración         | Loop Over Items | Procesamiento individual por ciudad.                    |
| Consulta API      | HTTP Request    | Respuesta de Open-Meteo, REST Countries u Open Library. |
| Transformación    | Code            | Datos normalizados y cálculos derivados.                |
| Consolidación     | Merge           | Unión de las tres ramas del workflow.                   |
| Salida            | Reporte Final   | JSON estructurado con estado, fuentes y errores.        |

#### D. Relación con el primer pipeline implementado

El primer pipeline académico no se elimina del análisis; se utiliza como antecedente técnico. En ese flujo se integraron Open-Meteo, REST Countries y Open Library. Cada rama tenía una fuente distinta, una transformación propia y un resultado que finalmente se consolidaba en un reporte JSON.

La rama de clima demuestra el uso de parámetros dinámicos y procesamiento por elementos. La rama de países demuestra cómo normalizar respuestas con estructuras anidadas. La rama

bibliográfica demuestra filtrado de resultados y manejo de errores básicos. Estos aprendizajes se trasladan al reto final, donde las fuentes son más institucionales y los formatos más variables.

El nodo Merge del primer reto anticipa la consolidación del reto final. En el nuevo escenario, el objetivo no es solo unir respuestas, sino construir un estado global de ejecución: fuentes correctas, fuentes fallidas, rutas generadas, archivos guardados y mensajes de control.

La diferencia principal es el nivel de responsabilidad. El primer reto produjo un reporte académico; el reto final debe producir insumos trazables para un tablero institucional. Por ello, se agregan validación más estricta, carpetas organizadas, logs, reintentos, última versión válida y posible carga en Oracle Database.

Esta relación muestra una progresión coherente: de un pipeline multi-fuente simple hacia una arquitectura RPA institucional. La experiencia inicial sirve para comprender el patrón antes de aplicarlo al macroentorno ecuatoriano.

Por esta razón, el documento conserva la evidencia del primer workflow, pero la interpretación se enfoca en el reto final. El primer reto es la base de aprendizaje; el reto final es el caso de aplicación principal.



Fig. 1. Workflow del primer reto implementado en n8n, utilizado como antecedente técnico del reto final.

#### E. Entregables y criterios de cumplimiento del reto

El reto final exige evidencias técnicas, no únicamente explicación teórica. Entre los entregables esperados se encuentran el diagrama de arquitectura, el workflow exportado en JSON, la estructura de carpetas con archivos descargados, el registro de errores y el historial de ejecuciones automáticas.

El diagrama de arquitectura se corrige mediante C4 Nivel 2 para mostrar contenedores generales y sistemas relacionados. Esta vista evita detallar cada nodo interno de n8n, porque ese

detalle corresponde al Nivel 3. Así se cumple la observación de presentar una arquitectura más general y profesional.

La estructura de carpetas debe organizar los archivos por institución y categoría. La nomenclatura debe incluir fuente, indicador y fecha, por ejemplo fuente\_indicador\_YYYYMMDD.ext. Este criterio permite recuperar versiones anteriores y comprobar qué archivo fue usado en una ejecución.

El registro de errores debe incluir fuente, fecha, estado, número de intentos, ruta de archivo si existe y mensaje de error.

Esta evidencia es necesaria para demostrar que el pipeline no solo funciona cuando todo sale bien, sino que también controla fallos.

### F. Salidas operativas esperadas

La primera salida operativa es el repositorio de archivos crudos. Este repositorio conserva los recursos descargados tal como fueron obtenidos o con una mínima organización por fuente. Su objetivo es permitir trazabilidad, revisión manual y respaldo ante fallos de procesamiento.

La segunda salida es el conjunto de logs de ejecución. Los logs permiten saber cuándo se ejecutó el flujo, qué fuente participó, si la descarga fue exitosa, qué ruta se generó y qué error ocurrió si la fuente falló. Estos campos provienen de la experiencia del primer reporte JSON, pero se adaptan al reto final.

La tercera salida es Oracle Database. En esta etapa se propone cargar datos normalizados en tablas staging por fuente. La carga puede implementarse de forma gradual, iniciando con archivos que tengan estructura más estable.

La cuarta salida es la notificación al equipo de Datos. La notificación no debe ser extensa; debe resumir fuentes procesadas, fallos parciales, rutas actualizadas y estado de carga. Este correo funciona como cierre operativo de cada ejecución.

En conjunto, estas salidas permiten que el pipeline sea verificable. Un evaluador puede revisar el archivo en carpeta, el log, el estado del workflow y la notificación generada.

Tabla III. Campos de control que se trasladan del reporte JSON inicial hacia los logs del reto final.

| Campo               | Propósito                                          |
|---------------------|----------------------------------------------------|
| version             | Identificar la versión lógica del pipeline.        |
| timestamp           | Registrar fecha y hora de ejecución.               |
| estado              | Indicar si la ejecución fue completa o incompleta. |
| fuentes             | Enumerar las APIs consultadas.                     |
| clima_regional      | Guardar estadísticas climáticas agregadas.         |
| contexto_pais       | Guardar indicadores normalizados de países.        |
| recursos_academicos | Guardar libros filtrados desde Open Library.       |
| errores             | Registrar fallos detectados por rama.              |

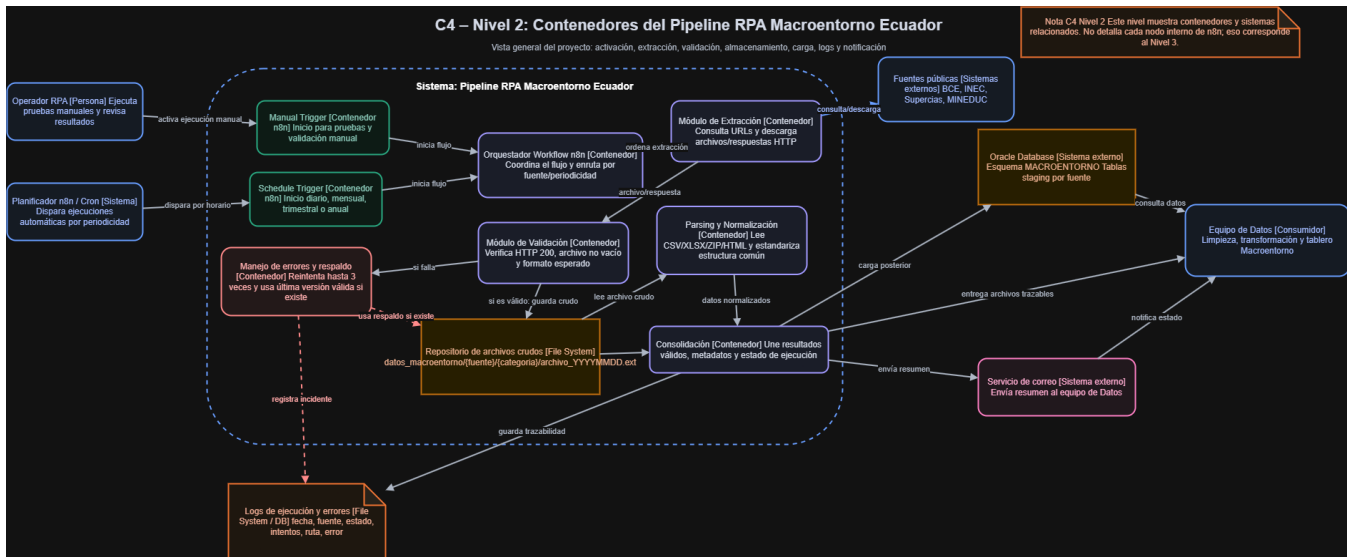


Fig. 2. Arquitectura C4 Nivel 2 del Pipeline RPA Macroentorno Ecuador: contenedores, fuentes, repositorios, logs y notificación.

## VI. ARQUITECTURA PROPUESTA

La arquitectura corregida se presenta mediante el modelo C4 Nivel 2. Este nivel muestra contenedores y sistemas relacionados sin detallar cada nodo interno de n8n. La decisión responde a la observación recibida: el diagrama debe ser más general y debe representar la solución completa del proyecto, no solo una captura del flujo.

En la vista C4 Nivel 2, el sistema principal es el Pipeline RPA Macroentorno Ecuador. Dentro de este sistema se ubican los contenedores funcionales: Manual Trigger, Schedule Trigger, Orquestador Workflow n8n, Módulo de Extracción, Módulo de Validación, Repositorio de archivos crudos, Parsing y Normalización, Consolidación, Manejo de errores y respaldo.

Fuera del sistema se representan los sistemas externos: fuentes públicas, Oracle Database, servicio de correo y equipo

de Datos. Esta separación permite entender qué elementos pertenecen al pipeline y qué elementos son dependencias externas o consumidores de resultados.

La arquitectura inicia con dos eventos posibles. El operador RPA activa manualmente el flujo durante pruebas, mientras que el planificador n8n o Cron dispara ejecuciones automáticas por horario. Ambos caminos llegan al orquestador, que coordina la ejecución según la fuente y periodicidad.

El módulo de extracción consulta las fuentes públicas mediante URLs y descarga archivos o respuestas HTTP. Luego, el módulo de validación verifica que el recurso cumpla condiciones mínimas. Si la validación es correcta, el archivo se almacena como crudo; si falla, se activa el manejo de errores y se registra el incidente.

Tabla IV. Organización técnica de fuentes por periodicidad y salida propuesta para el reto final.

| Periodicidad   | Fuentes                                         | Salida propuesta                   |
|----------------|-------------------------------------------------|------------------------------------|
| Diaria         | BCE petróleo WTI y riesgo país                  | Tablas Oracle + log de ejecución   |
| Mensual        | BCE sector real y Supercias directorio          | Esquema por fuente                 |
| Trimestral     | INEC ENEMDU                                     | Indicadores laborales normalizados |
| Anual/estática | BCE cuentas, Supercias ranking, MINEDUC y Censo | Datos históricos y registros base  |

| Criterio       | Pregunta de control                          |
|----------------|----------------------------------------------|
| Disponibilidad | ¿La fuente respondió correctamente?          |
| Integridad     | ¿El archivo o respuesta contiene datos?      |
| Formato        | ¿La extensión o estructura es la esperada?   |
| Coherencia     | ¿Existen campos u hojas obligatorias?        |
| Persistencia   | ¿Los datos se cargaron en Oracle DB?         |
| Notificación   | ¿Se informó el resultado al equipo de Datos? |

### A. Vista C4 Nivel 2: contenedores

El contenedor Manual Trigger representa la ejecución de prueba. Su evento es la acción del operador RPA al presionar Execute Workflow. Esta forma de activación es necesaria durante la construcción porque permite ejecutar una fuente específica, revisar el output y corregir errores de configuración.

El contenedor Schedule Trigger representa la automatización real. Su evento es una fecha y hora programada. En el reto final se puede configurar un schedule diario para fuentes de actualización diaria, mensual para fuentes mensuales, trimestral para ENEMDU y anual o bajo demanda para recursos estáticos.

El Orquestador Workflow n8n coordina la ejecución general. No se encarga de una tarea de negocio específica, sino de conectar los módulos. Desde el punto de vista C4, este contenedor representa el corazón del pipeline, porque recibe el evento de activación y ordena la extracción, validación, normalización y notificación.

El Repositorio de archivos crudos se modela como un File System porque guarda los recursos descargados. En la implementación con Docker, este repositorio se monta como volumen para que n8n pueda escribir dentro de /home/node/datos\_macroentorno y el usuario pueda revisar los archivos desde Windows.

### B. Manejo de errores, logs y continuidad

El manejo de errores se diseña como un contenedor propio porque no es un detalle secundario. Su responsabilidad es aplicar reintentos, registrar causa, conservar timestamp, guardar estado y utilizar la última versión válida si existe. Esta lógica evita que una fuente inestable detenga todo el pipeline.

Los logs de ejecución y errores se almacenan como archivo o base de datos. Cada registro debe incluir fuente, fecha, estado, intentos, ruta de archivo y mensaje de error. Esta información permite auditar la ejecución y preparar evidencia para la entrega del reto.

La continuidad operativa se basa en fallos parciales. Si una fuente falla después de los reintentos, el sistema notifica el incidente y continúa con el resto. Esta decisión es importante porque el pipeline consume fuentes externas sobre las que no se tiene control total.

## VII. MÉTODO DE TRABAJO

El método de trabajo propuesto es incremental. En lugar de intentar automatizar todas las fuentes al mismo tiempo, se construye primero un flujo base con una fuente de menor complejidad. Este flujo sirve para validar descarga, escritura en disco, nomenclatura, validación básica y registro de log.

La primera fase consiste en analizar las fuentes y clasificar periodicidades. En esta etapa se revisan URLs, formatos disponibles, frecuencia de actualización, institución responsable y dificultad técnica. El resultado es un inventario de fuentes que alimenta la configuración del workflow.

La segunda fase corresponde al diseño arquitectónico. Aquí se utiliza C4 Nivel 2 para representar contenedores y dependencias externas. La arquitectura permite verificar si el proyecto contempla activación, extracción, validación, almacenamiento, normalización, logs, Oracle y notificación.

La tercera fase es la implementación del flujo base en n8n. Se configuran nodos de activación, datos de fuente, HTTP Request, preparación de nombre de archivo, IF de validación y Write File to Disk. En entornos Docker se ajustan rutas internas y volúmenes compartidos.

La cuarta fase consiste en agregar manejo de errores. Se definen reintentos, salida false del IF, construcción de log de error y notificación de fallo parcial. Esta fase transforma el workflow de una prueba simple a un proceso más resistente.

La quinta fase amplía fuentes y periodicidades. Una vez probado el patrón, se replica para BCE, INEC, Supercias y MINEDUC, ajustando cada extracción según formato y reglas de validación. Finalmente, se documentan evidencias de ejecución, estructura de carpetas y resultados.

| Aspecto    | Primer reto                                               | Reto final                                              |
|------------|-----------------------------------------------------------|---------------------------------------------------------|
| Fuentes    | APIs públicas: Open-Meteo, REST Countries y Open Library. | Instituciones públicas: BCE, INEC, Supercias y MINEDUC. |
| Activación | Manual Trigger.                                           | Schedule Trigger por periodicidad.                      |
| Salida     | Reporte JSON consolidado.                                 | Carga en Oracle DB, trazabilidad y notificación.        |
| Errores    | Continuar salida regular y registrar error básico.        | Reintentos, logs, alerta y control por fuente.          |
| Alcance    | Práctica académica multi-fuente.                          | Pipeline institucional para equipo de Datos.            |

### A. Fases de implementación

La implementación se organiza por fases para reducir riesgo. La fase 1 valida una fuente simple y confirma que n8n puede guardar archivos desde Docker. La fase 2 incorpora logs y manejo de errores. La fase 3 agrega parsing o lectura de

archivos. La fase 4 prepara carga posterior en Oracle Database. La fase 5 automatiza schedules por periodicidad.

Cada fase debe dejar evidencia. La evidencia mínima incluye captura del workflow, archivo guardado en carpeta, JSON exportado, log generado, error controlado si aplica y explicación de cómo se disparó el trigger. Esta documentación es necesaria para demostrar cumplimiento del reto.

La estrategia de pruebas debe considerar casos correctos y casos de error. Un caso correcto verifica descarga y guardado. Un caso de error puede simular URL inválida, archivo vacío o falta de binary.data. El flujo debe responder con log y notificación, no con una interrupción silenciosa.

El método también contempla revisión con el equipo o tutor antes de avanzar a fuentes más complejas. Esta validación evita invertir tiempo en una arquitectura incorrecta y permite ajustar el alcance del pipeline de acuerdo con la complejidad real de las fuentes públicas.

### VIII. RESULTADOS ESPERADOS Y VALIDACIÓN

El primer resultado esperado es un workflow n8n exportable en formato JSON. Este archivo debe contener nodos con nombres descriptivos y conexiones claras. El flujo debe demostrar al menos una fuente funcional y servir como patrón para incorporar nuevas fuentes.

El segundo resultado esperado es una estructura de carpetas trazable. El repositorio debe organizarse por institución y categoría, por ejemplo datos\_macroentorno/supercias/ranking. Los archivos deben guardarse con fecha para evitar sobrescritura y permitir control de versiones.

El tercer resultado esperado es un registro de ejecución. El log debe mostrar fecha, fuente, estado, intentos, ruta y mensaje. Este registro permite comprobar que el pipeline se ejecutó y que el resultado no depende únicamente de una observación visual en n8n.

El cuarto resultado esperado es el manejo de errores. Si una fuente falla, el sistema debe registrar el incidente y continuar con el resto de fuentes cuando sea posible. En una versión avanzada, el flujo utilizará la última versión válida como respaldo.

El quinto resultado esperado es la comunicación al equipo de Datos. El correo o mensaje de notificación debe resumir archivos nuevos, fuentes correctas, fuentes fallidas, rutas actualizadas y tablas de Oracle afectadas si ya se implementó la carga.

La validación técnica se realiza comprobando que el trigger se ejecuta, que el HTTP Request descarga datos, que el archivo existe en la carpeta del volumen Docker, que el IF clasifica correctamente el estado y que los logs se escriben en la carpeta correspondiente.

La validación arquitectónica se realiza revisando si el diagrama C4 Nivel 2 representa todos los contenedores necesarios. Deben estar presentes activación, orquestación, extracción, validación, repositorio, normalización, consolidación, logs, errores, Oracle y notificación.

Con estos resultados, el proyecto queda alineado con el reto final porque no solo presenta teoría de RPA, sino una propuesta

directamente aplicable al Pipeline RPA Macroentorno Ecuador. La evidencia final debe mostrar que la arquitectura se convirtió en una guía de implementación y que cada decisión técnica responde a una necesidad del reto.

En síntesis, la proyección del pipeline consiste en convertir la arquitectura C4 en un conjunto de workflows mantenibles, observables y preparados para integración posterior. El reto no termina en descargar archivos; la meta es construir una base automatizada que reduzca tareas manuales, conserve trazabilidad, gestione fallos y entregue información confiable para el Tablero Macroentorno Ecuador.

El cuarto riesgo es intentar implementar todas las fuentes al mismo tiempo. La estrategia incremental reduce ese problema. Primero se valida una fuente de baja complejidad, como Supercias Ranking; luego se agregan logs; después se incorporan validaciones; y finalmente se replica el patrón hacia BCE, INEC y MINEDUC. Esta forma de trabajo permite avanzar con control técnico y facilita detectar errores antes de que el flujo crezca demasiado.

El tercer riesgo es la falta de evidencia. Si el flujo descarga un archivo pero no registra la ejecución, el resultado queda incompleto desde el punto de vista de auditoría. Por eso, los logs forman parte central de la arquitectura. El log debe ser tratado como salida del sistema, no como un elemento secundario. En el reto final, esta evidencia permitirá demostrar al tutor que el pipeline no solo funciona visualmente, sino que deja registros verificables.

El segundo riesgo es depender de una única ejecución exitosa. Para fuentes críticas, la arquitectura plantea reintentos controlados y uso de la última versión válida si existe. Esta decisión evita que un fallo temporal deje sin insumo al equipo de Datos. Sin embargo, el respaldo no debe ocultar el error; la notificación debe indicar que se usó una versión anterior y que la fuente actual no pudo procesarse correctamente.

El primer riesgo técnico es la variabilidad de las fuentes públicas. Un portal puede cambiar la ruta de descarga, modificar el nombre del archivo o entregar una página HTML en lugar de un archivo esperado. Para mitigar este riesgo, el pipeline debe validar no solo que exista una respuesta HTTP, sino que el contenido sea coherente con la extensión y estructura esperada. Si la validación falla, el flujo debe registrar el incidente y continuar con las demás fuentes cuando sea posible.

### D. Riesgos técnicos y mitigación

Cuando se incorpore Oracle Database, la carga debe diseñarse con cuidado para evitar duplicados. Si el flujo se ejecuta dos veces el mismo día, no debería insertar dos veces los mismos registros. Una estrategia posible es utilizar claves por fuente,



indicador, periodo y fecha de publicación. Otra opción es cargar primero en tablas staging y luego aplicar reglas de actualización controlada. Estas decisiones deben documentarse porque afectan directamente la calidad del repositorio institucional.

Otro punto relevante es el versionamiento. El workflow exportado en JSON debe guardarse como evidencia del avance del reto, pero también como respaldo técnico. Si una modificación rompe el pipeline, contar con versiones anteriores permite restaurar el flujo. Del mismo modo, los archivos descargados deben conservar fecha para distinguir una versión histórica de una versión nueva. El versionamiento aplica tanto al diseño de n8n como a los datos obtenidos.

La arquitectura también debe proteger credenciales y configuraciones sensibles. En una implementación real, las credenciales de Oracle Database y del servicio de correo no deben escribirse dentro de nodos Code ni en capturas del documento. Deben administrarse mediante credenciales internas de n8n o variables de entorno. Esta práctica reduce el riesgo de exposición accidental y permite migrar el flujo entre ambientes sin cambiar la lógica del workflow.

El uso de Docker introduce una consideración técnica importante: n8n se ejecuta dentro de un contenedor y no accede directamente a las rutas de Windows. Por eso, el diseño contempla un volumen compartido entre el sistema operativo anfitrión y la ruta interna /home/node/datos\_macroentorno. Esta configuración permite que n8n escriba archivos dentro del contenedor y que el usuario pueda revisarlos desde su carpeta local.

### C. Seguridad, despliegue y versionamiento

El diseño C4 Nivel 2 ayuda a comprender esta escalabilidad porque no representa cada nodo interno, sino contenedores funcionales. El módulo de extracción puede crecer con nuevas fuentes; el módulo de validación puede incorporar nuevas reglas; el repositorio puede ampliar categorías; y el módulo de notificación puede incluir más destinatarios. Esta visión general evita que la arquitectura quede atada a una sola implementación puntual.

La capa de abstracción propuesta en la arquitectura cumple precisamente esa finalidad. Su función es ocultar la complejidad de cada fuente y entregar una estructura común al resto del pipeline. Por ejemplo, aunque una fuente llegue como CSV, otra como XLSX y otra como HTML, el flujo debe producir metadatos uniformes: fuente, categoría, periodicidad, fecha de descarga, ruta del archivo, estado y observaciones. Esta estructura común facilita consolidar resultados y notificar el estado general del proceso.

Una mejora posterior consiste en separar responsabilidades mediante subworkflows. Un subworkflow puede encargarse de registrar errores, otro de validar archivos, otro de preparar metadatos y otro de cargar datos en Oracle. Esta separación evita duplicar lógica en cada fuente. Además, si se modifica la forma de registrar errores, se actualiza un único componente reutilizable en lugar de cambiar varios flujos independientes.

El reto final incluye fuentes con periodicidades diferentes: diarias, mensuales, trimestrales, anuales y estáticas. Si todas las reglas se implementan en un único flujo rígido, el mantenimiento se vuelve complejo. Por ello, la arquitectura debe permitir crecimiento incremental. El patrón recomendado es construir primero una fuente simple, validar su descarga, guardar el archivo, registrar el log y luego reutilizar esa estructura para otras fuentes.

### B. Escalabilidad y mantenimiento con subworkflows

La nomenclatura de archivos cumple una función de control. Un nombre con institución, categoría, indicador y fecha evita sobrescrituras y facilita ubicar versiones históricas. En el reto final, esta práctica se refleja en rutas como datos\_macroentorno/supercias/ranking y en archivos con fecha de descarga. Aunque parezca un detalle operativo, esta regla mejora la reproducibilidad del proceso y permite demostrar qué archivo alimentó una ejecución específica.

El gobierno de datos también exige conservar trazabilidad entre la fuente original y el dato utilizado posteriormente. Por esa razón, el repositorio de archivos crudos no debe eliminarse después de la carga en Oracle Database. Mantener el archivo original permite comparar versiones, revisar cambios de estructura y justificar el origen de los registros almacenados. Esta decisión es especialmente útil en fuentes públicas que pueden cambiar sus enlaces, formatos o criterios de publicación.

Este enfoque permite que el equipo de Datos no dependa de una revisión manual del workflow. Si una ejecución se completa correctamente, el log demuestra qué fuente fue procesada y qué archivo quedó disponible. Si una fuente falla, el registro permite identificar el punto exacto del problema y tomar decisiones sin repetir todo el proceso desde cero. De esta forma, el pipeline pasa de ser una automatización aislada a un proceso controlado y auditable.

La arquitectura propuesta no debe evaluarse únicamente por su capacidad de descargar archivos. En un escenario institucional, el valor principal del pipeline está en generar evidencia verificable de cada ejecución. Por ello, la observabilidad se convierte en un criterio técnico importante: cada fuente debe dejar constancia de la fecha de consulta, estado de respuesta,

ruta del archivo generado, número de intentos y mensaje de error si existió una falla.

#### A. Observabilidad y gobierno de datos

### IX. DISCUSIÓN TÉCNICA Y PROYECCIÓN DEL PIPELINE

#### X. CONCLUSIONES

El documento corregido centra el análisis en el reto final y conserva el primer pipeline únicamente como antecedente técnico. Esta organización mejora la coherencia porque el estado del arte, el marco conceptual, la arquitectura y el método de trabajo se orientan al Pipeline RPA Macroentorno Ecuador.

El modelo C4 Nivel 2 resulta adecuado para describir la arquitectura porque muestra contenedores y sistemas relacionados sin caer en el detalle de cada nodo interno de n8n. De esta manera, el diagrama cumple una función arquitectónica y no solo operativa.

Integrar las herramientas dentro del marco conceptual permite explicar por qué se usan n8n, Docker, HTTP Request, Code, Write File to Disk, Oracle Database, logs y notificación. Estas herramientas no aparecen como una lista aislada, sino como conceptos que sostienen la solución propuesta.

La descripción del reto final aclara el alcance del proyecto: extraer datos públicos de instituciones ecuatorianas, validar recursos, guardar archivos trazables, manejar errores, cargar datos posteriormente en Oracle y notificar al equipo de Datos. Esta sección diferencia claramente el problema institucional del primer reto académico.

El método de trabajo incremental reduce riesgo técnico. Implementar primero una fuente simple permite validar rutas, binarios, escritura en disco y logs antes de ampliar el flujo hacia fuentes más complejas. Esta estrategia es adecuada para un reto de varias semanas.

Como trabajo pendiente queda completar la implementación de todas las fuentes, configurar schedules por periodicidad, registrar historial de ejecuciones automáticas y preparar la carga en Oracle Database. Con ello, la arquitectura documentada podrá convertirse en un pipeline operativo y evaluable.

#### XI. REFERENCIAS

- [1] R. Syed et al., "Robotic Process Automation: Contemporary themes and challenges", *Computers in Industry*, vol. 115, 2020.
- [2] n8n, "Workflow automation platform and node documentation", n8n Docs, 2026.
- [3] n8n, "HTTP Request, Code, Schedule Trigger and file system nodes", n8n Docs, 2026.
- [4] Docker, "Volumes and bind mounts documentation", Docker Docs, 2026.
- [5] Oracle, "Oracle Database concepts and staging tables", Oracle Documentation, 2026.

- [6] S. Brown, "The C4 model for visualising software architecture", C4Model.com, 2026.
- [7] DGTITD-UTPL, "Pipeline de extracción automatizada de datos públicos del macroentorno ecuatoriano: reto de 7 semanas", Universidad Técnica Particular de Loja, 2026.
- [8] A. Robles, "Primer Reto: workflow de integración con Open-Meteo, REST Countries y Open Library", archivo JSON de n8n, 2026.
- [9] R. Syed, S. Suriadi, M. Adams, W. Bandara, S. J. J. Leemans, C. Ouyang, A. H. M. ter Hofstede, I. van de Weerd, M. T. Wynn, and H. A. Reijers, "Robotic Process Automation: Contemporary themes and challenges," *Computers in Industry*, vol. 115, Art. no. 103162, 2020, doi: 10.1016/j.compind.2019.103162.
- [10] J. Wewerka and M. Reichert, "Robotic Process Automation -- A Systematic Literature Review and Assessment Framework," arXiv:2012.11951, 2020, doi: 10.48550/arXiv.2012.11951.
- [11] D. A. da Silva Costa, H. S. Mamede, and M. M. da Silva, "Robotic Process Automation (RPA) adoption: a systematic literature review," *Engineering Management in Production and Services*, vol. 14, no. 2, pp. 1-12, 2022, doi: 10.2478/emj-2022-0012.
- [12] N. M. El-Gharib and D. Amyot, "Robotic Process Automation Using Process Mining -- A Systematic Literature Review," *Data & Knowledge Engineering*, Art. no. 102229, 2023, doi: 10.1016/j.datak.2023.102229.
- [13] H. Foidl, V. Golendukhina, R. Ramler, and M. Felderer, "Data Pipeline Quality: Influencing Factors, Root Causes of Data-related Issues, and Processing Problem Areas for Developers," arXiv:2309.07067, 2023.
- [14] C. Haase, T. Roeseler, and M. Seidel, "METL: a modern ETL pipeline with a dynamic mapping matrix," arXiv:2203.10289, 2022.
- [15] n8n, "Schedule Trigger node documentation," n8n Docs, 2026.
- [16] n8n, "HTTP Request node documentation," n8n Docs, 2026.
- [17] n8n, "Code node documentation," n8n Docs, 2026.
- [18] S. Brown, "Container diagram," The C4 Model for Visualising Software Architecture, 2026.